- "What techniques can you use to handle errors gracefully within such a composed functional pipeline?"
- "How would you implement a lazy-evaluated pipeline (like generators) instead of eagerly processing all data?"
- "What challenges arise when integrating immutable functional pipelines with class-based modules that encapsulate mutable state?"
- "Can you refactor this pipeline to support parallel processing of transformations?"

3.3 Asynchronous behavior in JavaScript

Mastering JavaScript's asynchronous behavior is essential for technical interviews. Modern web applications depend on asynchronous programming to stay responsive during operations like API requests, animations, or user interactions.

Because JavaScript is single-threaded and executes code synchronously by default, long-running tasks can block the main thread and freeze the UI. To address this, JavaScript provides asynchronous patterns such as callbacks, Promises, and `async`/`await`. Combined with the event loop, these techniques allow tasks to be scheduled and processed without blocking the application, enabling smooth and interactive user experiences.

Table 3.3 is Jane's review guide for asynchronous behavior. Use it as a checklist to ensure you are ready to tackle both conceptual and practical challenges in interviews with confidence.

Table 3.3 Key features of asynchronous behavior

Topic	Fundamentals	Applied skills	Advanced
Asynchronous programming	Explain the event loop, use callbacks, and describe Promises.	Chain Promises, use `async`/`await`, and handle asynchronous errors.	Optimize async flows, debug race conditions, and design async-heavy systems.